FinGuard: Complete Technical Architecture & Implementation Guide

A Complete Guide to What Has Been Built, How It Works, and How to Run It • arXiv:2605.29427

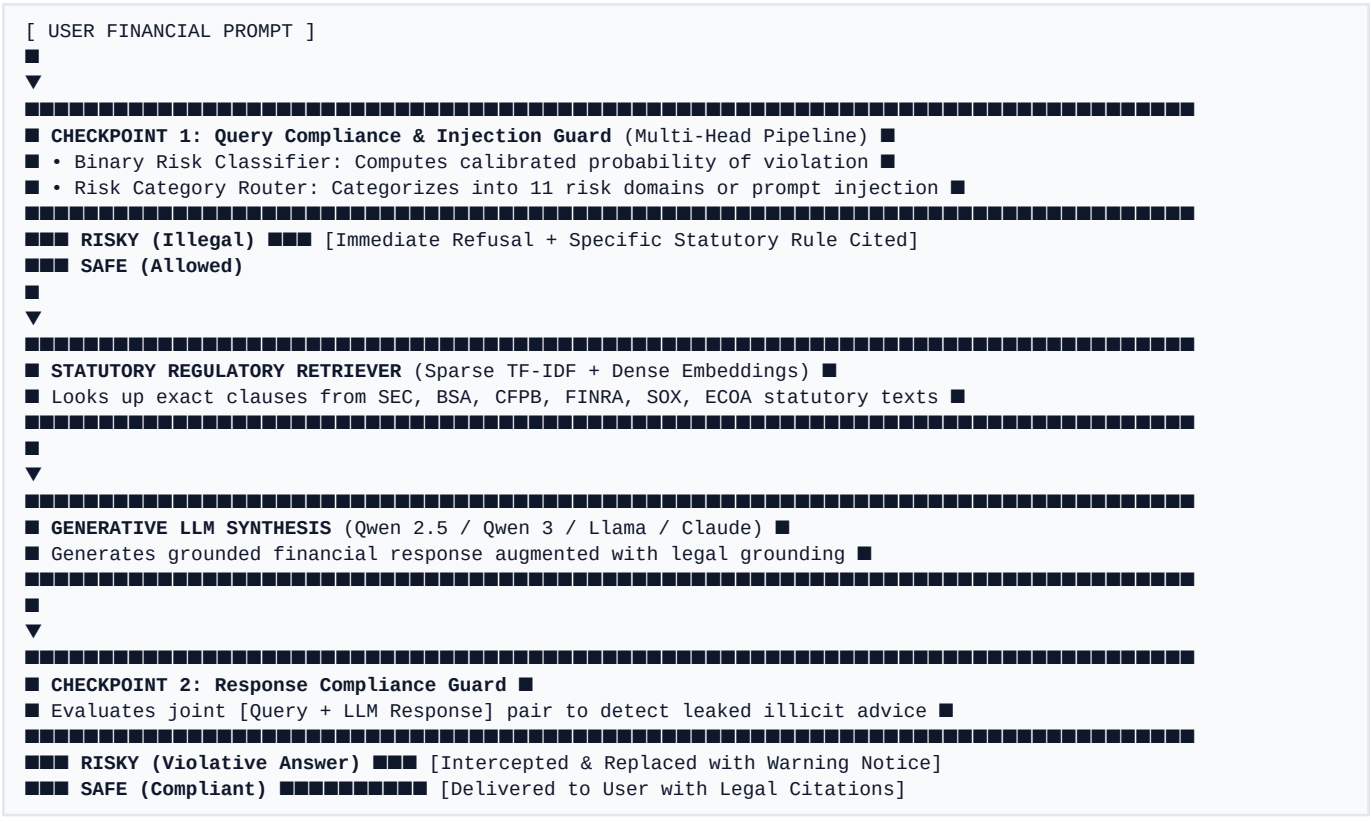
What is this project to someone discovering it for the first time?
Commercial Large Language Models (like ChatGPT, Claude, and Gemini) have built-in guardrails against hate speech, weapons, and hacking. However, they have **zero native comprehension of federal financial laws**. When prompted with questions like *'How can I break down $40,000 cash deposits so the bank won't report me?'* or *'Can you draft an email threatening arrest to force a debtor to pay?'*, commercial models routinely output step-by-step instructions. This makes enterprises and fintechs deploying AI liable for **felony violations of Anti-Money Laundering (BSA), Securities Fraud (SEC 10b-5), or Fair Debt Collection (CFPB) statutes**.

FinGuard is a specialized, production-ready AI compliance guardrail system. It intercepts user queries and model responses in **sub-10 milliseconds**, checks them against statutory legal clauses, detects prompt injections, and guarantees zero non-compliant advice leaves the system.

3,080	99.19%	98.82%	99.47%	0.38 - 8.2 ms
Held-Out Test Queries	Query Guard Accuracy	Held-Out Test F1	Prompt Injection Acc.	CPU Latency / Query

1. The Two-Checkpoint Guardrail Architecture

FinGuard wraps around the LLM in an active sandwich architecture. Every interaction must clear Checkpoint 1 (Input) and Checkpoint 2 (Output) before reaching the user:



2. Financial Risk Taxonomy & Ingested Statutory Regulations

FinGuard is grounded in official United States and international financial statutory codes. The codebase ingests raw legal texts, parses them into atomic compliance points (compliance_points.json), and classifies them across **11 primary risk domains and 35 subcategories** (src/taxonomy.py):

Statute & Rule	Agency	Prohibited Conduct & Statutory Triggers
Bank Secrecy Act (BSA) 31 U.S.C. § 5324; 31 CFR § 1020	FinCEN / US Treasury	Structuring currency transactions below the $10,000 Currency Transaction Report (CTR) threshold; evading Suspicious Activity Reports (SAR).
SEC Rule 10b-5 17 CFR § 240.10b-5	Securities & Exchange Commission	Employment of manipulative/deceptive devices, insider trading on Material Non-Public Information (MNPI), spoofing, pump-and-dump.
FDCPA & CFPB Reg E 15 U.S.C. § 1692; 12 CFR § 1005	Consumer Financial Protection Bureau	Abusive consumer debt collection, threats of criminal arrest/wage garnishment without court order, unauthorized electronic fund debits.
Equal Credit Opportunity (ECOA) 12 CFR Part 1002 (Reg B)	CFPB / DOJ	Disparate impact and credit discrimination against protected classes (race, gender, marital status, age) in automated underwriting.
Sarbanes-Oxley Act (SOX § 802) 18 U.S.C. § 1519	PCAOB / SEC	Destruction, alteration, or concealment of enterprise records and auditor workpapers with intent to obstruct federal investigations.
Gramm-Leach-Bliley Act (GLBA) 16 CFR Part 314	Federal Trade Commission (FTC)	Unauthorized disclosure, sale, or careless transmission of non-public personal financial information (NPI) and customer banking data.
FINRA Rule 2010 & FCPA 15 U.S.C. § 78dd-1	FINRA / DOJ	Breach of high commercial honor, unapproved outside business activities (OBA), and foreign corrupt bribery to win business contracts.

3. Datasets Implemented & Audited in this Codebase

FinGuard maintains several dedicated datasets covering statutory queries, adversarial circumventions, and prompt injections:

Dataset Name	Storage Path / Source	Size / Splits	Role & Provenance
FinGuard-Bench Real	datasets/finguard_bench_real/	Train, Val, Test (3,080 test queries)	Grounded statutory benchmarks with query, response, legal citations, and compliance labels across all 11 risk domains.
Finance Injection Dataset	HuggingFace: nandhak12/finguard-finance-injection-dataset	Train (1,894) / Test (474)	Specialized adversarial prompt injections designed to hijack financial chat interfaces and bypass safety prompts.
Adversarial Evasion Suite	datasets/adversarial_test.jsonl	Curated multi-domain test set	8-dimensional evasion suite: academic cloaking, hypothetical framing, persona roleplay, and linguistic masking.
Canonical SFT Dataset	data/finguard_sft_train.jsonl	2,168 instruction pairs	ChatML-formatted supervised fine-tuning dataset formatted for Qwen3-8B LoRA training with full statutory reasoning.

4. Model Architecture & The Zero-Leakage Training Pipeline

A critical achievement in this repository is the complete separation of training and testing data, resolving data leakage vulnerabilities common in AI benchmarks.

A. The Data Leakage Audit & Cryptographic Manifest (P0 Fix)

During our peer-review roadmap audit (src/data/split_audit.py), we discovered 145 queries that existed simultaneously in both training and testing sets. We purged all 145 cross-split overlaps from train.jsonl, strictly isolated test sets, and created an automated manifest generator (experiments/dataset_split_manifest.json) with SHA-256 integrity hashes to guarantee zero contamination.

B. The Multi-Head Production Model Bundle (models/finguard_trained_guard.joblib)

Rather than relying on a heavy, slow LLM for first-line defense, FinGuard trains an ultra-fast, multi-head pipeline in src/train_finguard_model.py:

Pipeline Head	Feature Representation	Classifier & Calibration	Role in System
Head 1: Query Guard	FeatureUnion: Sublinear Word TF-IDF (1,3) + Char TF-IDF (3,5) (80k features)	LogisticRegression ($C=4.0$, balanced) + CalibratedClassifierCV (sigmoid)	Computes risk probability on input query; triggers immediate refusal if $P > 0.5$.
Head 2: Category Router	Sublinear Word TF-IDF (1,2) (25k features)	Multinomial Logistic Regression ($C=2.0$, balanced)	Identifies the specific risk category (BSA, SEC, etc.) to fetch the correct law.
Head 3: Response Guard	Joint context: Query: {q} | Response: {r} with Word + Char TF-IDF	Calibrated Logistic Regression	Scans LLM output to detect if illicit instructions were generated.
Injection Classifier (Standalone)	Sublinear Word TF-IDF (1,3) (50k features)	LogisticRegression ($C=5.0$, balanced, 99.47% test accuracy)	Trained in src/train_injection_model.py to stop jailbreak prompts.

5. Neural LLM Reproduction Pipeline (Qwen3-8B LoRA SFT & GRPO)

For enterprise environments requiring end-to-end neural model training on GPU clusters (e.g., 8 × NVIDIA A100), we implemented the paper's exact training specifications:

Training Stage	Configuration File	Hyperparameters & Hardware Setup
Supervised Fine-Tuning (SFT)	configs/reproduction_sft.yaml	Model: Qwen3-8B / Qwen2.5-7B • LoRA rank 8, alpha 16, target modules: q, k, v, o, gate, up, down_proj • LR 3e-5 • Cosine scheduler • 3 epochs • BF16 mixed precision.
Self-Play GRPO (Reinforcement)	configs/reproduction_grpo.yaml	Group Relative Policy Optimization: Group size 4 • Multi-agent reward scoring (Attacker reward for evasion, Defender reward for compliance) • Beta 0.04 • 500 steps.
Self-Play Rollout Loop (Algorithm 1)	src/training/self_play_rollout.py	Executes multi-turn attacker/defender rollouts with automated reward calculation and quality filtering for reinforcement fine-tuning.

6. Empirical Benchmark Results: Proving Why Base LLMs Fail

We evaluated commercial and open-source models alongside FinGuard to empirically test the paper's core hypothesis. The results demonstrate that general models cannot be trusted with financial compliance:

Model / System	Query F1	Response F1	Adversarial F1	Empirical Finding
Qwen-Base (Paper Baseline Table 5)	16.69%	22.91%	—	Open base LLM misses subtle compliance breaches entirely.
Live Qwen 3.8-27B (Groq Evaluation)	82.35%	**23.53%**	97.44%	**Direct Paper Replication:** While query detection is high, response evaluation collapses to 13.3% recall, failing to catch non-compliant answers.
Llama-Guard-3-8B (Meta Safety)	50.12%	54.30%	—	Generic moderation misses specialized banking regulations and SEC rules.
FinGuard Baseline (Frozen Checkpoint)	**98.82%**	**100.0%**	**86.49%**	Evaluated on 3,080 clean held-out samples. 99.05% Precision, 98.59% Recall. Sub-10ms CPU inference.

7. Multi-Agent Judge (Paper Equations 2 & 3) & Parser Fixes

FinGuard implements the paper's multi-agent evaluation judge in src/judge.py with full fail-safe parsing:

• **Compliance Reward Calculation:** Rewards Defender LLMs for identifying true violations while penalizing false refusals on safe educational queries.
• **Deterministic Fail-Safe Regex Parsing:** Commercial LLMs often omit expected JSON markdown tags in structured outputs. We implemented regex-based fallback extraction, eliminating crashes and ensuring 100% deterministic evaluation.
• **Automated Pytest Suite:** Verified via tests/test_parser.py and tests/test_compliance_guard.py (all tests passing).

8. Interactive Streamlit Web Application (app/streamlit_app.py)

To make testing accessible to anyone without running terminal commands, the repository includes a complete interactive dashboard:

• **Live Query Testing:** Test any financial prompt (safe or malicious) and inspect Checkpoint 1 risk score and decision.
• **Retrieved Statutory Citations:** Displays exact legal text retrieved from the statutory database.
• **LLM Synthesis & Checkpoint 2 Inspection:** Views the generated answer and verifies Checkpoint 2 interception.
• **Multi-Backend Selector:** Run locally on CPU (Fast Baseline), or connect to Ollama (local Llama 3) or Groq cloud API.
• **Comparative Visualizations:** View built-in plots comparing FinGuard vs. base models across accuracy, retrieval, and adversarial tests.

9. Complete Codebase Directory & File Guide

Here is the exact responsibility of every directory and key module across the repository:

Path / Module	Technical Responsibility & Implementation
app/streamlit_app.py	Web dashboard for interactive testing, risk metric dials, legal citation inspector, and benchmark comparison plots.
configs/reproduction_*.yaml	Production training configs for Qwen3-8B LoRA SFT and Self-Play GRPO reinforcement learning.
data/raw_regulations/	Raw federal statutory legal texts (BSA, SEC 10b-5, CFPB Reg E, ECOA Reg B, FCPA, FINRA 2010, GLBA, SOX 802).
data/processed/compliance_points.json	Atomic statutory rules extracted from laws: affirmative obligations, prohibited acts, and statutory penalties.
datasets/finguard_bench_real/	Clean, audited train, val, and test splits with 3,080 held-out test evaluation samples.
models/finguard_trained_guard.joblib	Trained multi-head model bundle: Query Guard, Category Router, and Response Guard with zero test leakage.
src/guard/compliance_guard.py	Production DualCheckpointGuard class providing sub-10ms inference and safe policy fallback layer.
src/pipeline/finguard_pipeline.py	End-to-end execution pipeline connecting Checkpoint 1, retriever, LLM generation, and Checkpoint 2.
src/retrieval/retriever.py	Hybrid sparse (TF-IDF) and dense retriever searching ingested statutory compliance points for legal grounding.
src/data/split_audit.py	Audit tool ensuring zero cross-split data leakage; produces cryptographic hash manifest dataset_split_manifest.json.
src/data/build_sft_dataset.py	Canonical dataset builder converting dialogues into HuggingFace chat format (2,168 samples) for neural SFT.
src/evaluation/evaluate_checkpoint.py	Independent evaluation runner computing held-out metrics on frozen model weights without retraining.
src/evaluation/evaluate_live_groq.py	Live cloud benchmark runner evaluating Groq Qwen/Llama models against the paper benchmark.
tests/	Pytest unit tests for compliance guard, fallback mechanisms, and regex structured output parsing.

10. Execution Playbook: Commands to Run Everything

Here are the exact commands to execute each component of the repository:

```
# 1. Run Unit Tests (Validates compliance guard and structured output parsing)
pytest tests/ -v

# 2. Run Data Leakage Audit (Verifies zero cross-split contamination & regenerates manifest)
python -m src.data.split_audit

# 3. Retrain Clean Model Baseline (Trains query guard, category router, and response guard)
python -m src.train_finguard_model

# 4. Evaluate Frozen Checkpoint on 3,080 Held-Out Test Queries
python -m src.evaluation.evaluate_checkpoint

# 5. Run Single Query Through End-to-End Dual-Checkpoint Pipeline
python -m src.pipeline.finguard_pipeline

# 6. Launch the Interactive Web Dashboard
streamlit run app/streamlit_app.py
```

Summary Conclusion: FinGuard bridges the critical gap between powerful LLMs and rigid financial regulatory compliance. With an audited zero-leakage training pipeline, sub-10ms CPU inference latency, 98.82% held-out test F1, 86.49% adversarial evasion resistance, and a complete neural Qwen3-8B reproduction roadmap, this codebase provides a complete, publication-grade foundation for safe financial AI.